

CV4: Task Planner v Jetpack Compose - Teória

1 Cieľ cvičenia

V tomto cvičení sa budeme venovať vedomému rozdeleniu väčšej obrazovky na menšie a znovupoužiteľné composable funkcie. Po absolvovaní cvičenia by ste mali rozumieť tomu, prečo nestačí napísať celú obrazovku do jednej veľkej composable funkcie a ako sa v Jetpack Compose navrhujú komponenty s jasnou zodpovednosťou.

Výsledkom bude jednoduchá obrazovka plánovača úloh. Používateľ uvidí hlavičku obrazovky, súhrnné karty, filtre a samotné karty úloh. Súčasťou obrazovky bude aj jednoduchý panel s detailom vybranej úlohy. Toto zadanie je vhodné práve preto, že jedna väčšia obrazovka už obsahuje viac logických blokov a prirodzene si pýta rozdelenie na menšie časti.

2 Composable decomposition

Pojmom *composable decomposition* označujeme rozdelenie väčšej obrazovky na menšie composable funkcie. Dôvodom nie je len kratší súbor, ale najmä oddelenie zodpovedností. Každá časť obrazovky má mať vlastný význam. V našom prípade môže mať jedna composable funkcia na starosti hlavičku, iná sumárne štatistiky, ďalšia filtre a ďalšia jednu konkrétnu kartu úlohy.

Ak by sme všetko písali do jednej funkcie, kód by sa rýchlo stal neprehľadným. Ťažšie by sa menil, horšie by sa testoval a vy ako programátor by ste ťažko rozlišovali, ktoré časti obrazovky sa opakujú a ktoré sú jedinečné.

3 Znovupoužiteľné composables

Znovupoužiteľný composable je taký komponent, ktorý nie je naviazaný na jeden jediný konkrétny prípad. Namiesto toho prijíma údaje cez parametre a svoju prácu vykonáva všeobecnejšie. To znamená, že rovnakú composable funkciu môžeme použiť viackrát s rôznymi dátami.

Dobrým príkladom je nasledujúca karta úlohy. Namiesto toho, aby sme pre každú úlohu ručne písali rovnaký blok používateľského rozhrania, vytvoríme composable funkciu `TaskCard(...)` a do nej budeme posilať objekt úlohy a callbacky pre interakciu.

```

TaskCard(
    task = task,
    onCompletedChange = { isCompleted ->
        onTaskCompletedChange(task.id, isCompleted)
    },
    onOpenDetailsClick = {
        onOpenDetailsClick(task.id)
    }
)

```

Tento princíp je v Jetpack Compose veľmi dôležitý. Komponenty sa tak stávajú čitateľnejšími a jednoduchšie sa rozširujú.

4 Parent composable a child composables

Vo väčších obrazovkách zvyčajne jedna vyššia composable funkcia drží hlavný stav a skladá celú obrazovku. Nižšie composables potom dostávajú len tie údaje a callbacky, ktoré naozaj potrebujú. Tento vzťah sa často opisuje ako parent a child composables.

V našom cvičení bude parent composable funkcia `TaskPlannerScreen()`. Práve ona bude držať zoznam úloh, vybraný filter, prepínač zobrazenia dokončených úloh a identifikátor práve vybratej úlohy. Nižšie composables budú tento stav iba zobrazovať alebo vracať akcie späť.

Takéto riešenie je dôležité nielen pre čitateľnosť. Neskôr sa veľmi prirodzene prepája s `ViewModelom`, kde hlavný stav už nebude uložený priamo v composable funkcii, ale mimo nej.

5 State hoisting v praxi

State hoisting znamená, že stav nedržíme vo vnútri každého malého komponentu, ale posúvame ho vyššie a nižšie komponenty dostanú iba hodnoty a callbacky. Vďaka tomu komponent zostáva jednoduchý a znovupoužiteľný.

Napríklad filter tlačidlá nebudú samy rozhodovať, ktorý filter je aktívny. Túto informáciu dostanú cez parameter `selected`. Po kliknutí iba zavolajú callback `onClick()`.

```

FilterButton(
    text = "High",
    selected = selectedFilter == TaskFilter.HIGH,
    onClick = { onFilterSelected(TaskFilter.HIGH) }
)

```

Podobne ani karta úlohy nebude sama meniť globálny stav aplikácie. Po kliknutí iba odošle informáciu späť vyššej vrstve.

6 Význam jednej opakujúcej sa entity

Toto cvičenie je postavené okolo jednej hlavnej entity, ktorou je `task`. Ak by obrazovka obsahovala priveľa nesúvisiacich blokov, videli by ste síce viacero komponentov, ale horšie by ste chápali, čo je hlavný opakujúci sa vzor.

Tu je opakujúca sa štruktúra jasná. Jedna úloha sa zobrazí cez jednu kartu. Viac úloh tvorí sekciu. Sekcie spolu vytvárajú celú obrazovku. Takéto usporiadanie veľmi dobre pripravuje pôdu pre ďalšie cvičenie, kde sa tie isté karty úloh budú zobrazovať v `LazyColumn`.

7 Summary cards a sekcie obrazovky

Okrem hlavnej entity úlohy je vhodné ukázať aj menšie pomocné komponenty. V tomto cvičení budú dobrým príkladom `SummaryCard` a `SectionCard`. Prvá composable funkcia slúži na zobrazenie jednej malej štatistiky, napríklad počtu aktívnych úloh. Druhá slúži ako univerzálny obal pre sekcie obrazovky, napríklad pre súhrn, filtre alebo samotný zoznam úloh.

Takéto komponenty sú dôležité, pretože ukazujú, že reusable composables nemusia byť iba položky zoznamu. Znovupoužiteľným komponentom môže byť aj obal sekcie alebo malý informačný blok.

8 Interakcie, ktoré majú viditeľný výsledok

Aby miniaplikácia nepôsobila ako čisto statická ukážka, je dôležité pridať interakcie, pri ktorých používateľ okamžite vidí zmenu. V našom prípade bude mať zmysel meniť filter priorít, skrývať dokončené úlohy, označiť úlohu ako hotovú a otvoriť detail vybratej úlohy.

Tieto akcie majú viacero výhod. Uvidíte, že stav ovplyvňuje viacero častí obrazovky naraz. Ak označí úlohu ako hotovú, zmení sa karta úlohy aj sumárne počítadlá. Ak otvorí detail úlohy, na obrazovke sa zobrazí nový panel. Takéto správanie už veľmi pekne demonštruje deklaratívny prístup `Compose`.

9 Dátový model

Aj v tomto cvičení je vhodné mať samostatný dátový model. Pre úlohy môžeme vytvoriť dátovú triedu `TaskItem`. Tá bude obsahovať identifikátor, názov úlohy, kategóriu, termín, prioritu, popis a informáciu o dokončení.

```
data class TaskItem(  
    val id: Int,  
    val title: String,  
    val category: String,  
    val dueDate: String,  
    val priority: TaskPriority,  
    val description: String,  
    val completed: Boolean = false  
)
```

Dátový model oddeľuje údaje od samotného UI a uľahčuje posielanie informácií medzi composables.

10 Prečo zatiaľ nepoužívame LazyColumn

Aj keď by sa karty úloh veľmi prirodzene dali zobrazovať cez `LazyColumn`, v tomto cvičení sa tomu zatiaľ zámerne vyhneme. Hlavnou témou CV4 nie sú zoznamy, ale rozdelenie obrazovky a návrh reusable komponentov.

Preto je v tejto fáze úplne v poriadku použiť obyčajnú `Column` s `verticalScroll`. V nasledujúcom cvičení sa tá istá myšlienka posunie ďalej a naučíte sa, ako tie isté opakujúce sa prvky efektívnejšie zobrazovať v lazy zozname.

11 Zhrnutie

Po absolvovaní tohto cvičenia by ste mali rozumieť tomu, ako sa v Jetpack Compose navrhuje väčšia obrazovka z menších composable funkcií. Mali by ste vedieť rozpoznať opakujúce sa časti používateľského rozhrania, vytvoriť z nich reusable komponenty, odovzdávať údaje a callbacky cez parametre a udržiavať hlavný stav vo vyššej composable funkcii.

Toto cvičenie zároveň vytvára veľmi dobrý základ k pochopeniu ďalších tém. V ďalšom kroku sa rovnaké karty úloh môžu zobraziť v `LazyColumn`. Neskôr sa stav môže presunúť do `ViewModelu` a údaje sa môžu začať ukladať cez `DataStore` alebo `Room`.